
Ingénierie Informatique
2^{ème} Année Cycle Ingénieurs
Filière : Développement Informatique

Projet de Fin d'Année (PFA)

**Application Mobile Intelligente de
Découverte et Gestion de Films**
MoviesApp

*Application Android intégrant l'Intelligence Artificielle Google Gemini,
l'API TMDB, Firebase Authentication et Google Maps*

Réalisé par :

Yassine Jaabouk

Encadré par :

M. Mohammed Bousmah

Année Universitaire : 2025 / 2026

Dédicaces

À mes chers parents,

*pour leur soutien indéfectible, leur amour inconditionnel
et leurs sacrifices tout au long de mon parcours.*

À mes professeurs et encadrants,

*pour leur guidance, leur patience et leur dévouement
à transmettre le savoir.*

À mes amis et collègues,

pour leur encouragement et leur présence précieuse.

Je dédie ce travail.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'année.

Je remercie tout d'abord **M. Mohammed Bousmah**, mon encadrant pédagogique, pour ses précieux conseils, sa disponibilité et son accompagnement tout au long de ce projet. Ses orientations techniques et académiques ont été déterminantes dans la réussite de ce travail.

Je remercie également **l'École Marocaine des Sciences de l'Ingénieur (EMSI) — Campus Casablanca Maarif** pour la qualité de la formation dispensée et les moyens mis à disposition pour mener à bien ce projet.

Mes remerciements vont aussi à l'ensemble du corps professoral de la filière Développement, dont l'enseignement rigoureux a constitué le socle de mes compétences techniques.

Enfin, je remercie ma famille et mes proches pour leur soutien moral constant et leurs encouragements durant toute ma formation.

Résumé

Ce projet de fin d'année, réalisé dans le cadre de la 4ème année — 2ème année du cycle ingénieur en Développement à l'EMSI Casablanca Maarif, porte sur la conception et le développement d'une application mobile Android intelligente dédiée à la découverte et à la gestion de films, intitulée **MoviesApp**.

L'application répond à un besoin croissant des utilisateurs de disposer d'une plateforme centralisée permettant de rechercher des films, de gérer une liste de films vus, de recevoir des recommandations personnalisées basées sur l'intelligence artificielle, et de localiser les cinémas à proximité.

Sur le plan technique, l'application est développée en **Java** sous **Android Studio**, avec un backend **FastAPI** (Python), une base de données **SQLite**, et une authentification via **Firebase Authentication**. L'intégration de l'API **TMDB** permet d'accéder à une base de données cinématographique riche et à jour. L'intelligence artificielle est assurée par l'API **Google Gemini**, qui alimente trois fonctionnalités clés : la recherche par vibe, les recommandations basées sur les films vus, et la détection de l'humeur par reconnaissance faciale via la caméra.

Les résultats obtenus montrent une application fonctionnelle, intuitive et moderne, intégrant des technologies avancées telles que la géolocalisation, la reconnaissance d'image par IA, et une interface utilisateur soignée inspirée des grandes plateformes de streaming.

Mots-clés : Application mobile Android, Intelligence Artificielle, TMDB API, Firebase, FastAPI, Google Gemini, Recommandation de films, Géolocalisation.

Abstract

This end-of-year project, carried out as part of the 4th year — 2nd year of the Engineering Cycle in Software Development at EMSI Casablanca Maarif, focuses on the design and development of an intelligent Android mobile application dedicated to movie discovery and management, entitled **MoviesApp**.

The application addresses the growing need for users to have a centralized platform to search for movies, manage a watched list, receive AI-powered personalized recommendations, and locate nearby cinemas.

Technically, the application is developed in **Java** using **Android Studio**, with a **FastAPI** (Python) backend, **SQLite** database, and authentication via **Firebase Authentication**. The integration of the **TMDB API** provides access to a rich and up-to-date movie database. Artificial intelligence is powered by the **Google Gemini API**, which drives three key features : vibe-based search, recommendations based on watched movies, and mood detection through facial recognition via the camera.

The results demonstrate a functional, intuitive, and modern application integrating advanced technologies such as geolocation, AI-based image recognition, and a polished user interface inspired by major streaming platforms.

Keywords : Android Mobile Application, Artificial Intelligence, TMDB API, Firebase, FastAPI, Google Gemini, Movie Recommendation, Geolocation.

Liste des Abréviations

Abréviation	Signification
AI / IA	Artificial Intelligence / Intelligence Artificielle
API	Application Programming Interface
TMDB	The Movie Database
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
REST	Representational State Transfer
SDK	Software Development Kit
UI	User Interface
UX	User Experience
GPS	Global Positioning System
IDE	Integrated Development Environment
SQL	Structured Query Language
ORM	Object Relational Mapping
MVP	Minimum Viable Product
CI/CD	Continuous Integration / Continuous Deployment
APK	Android Package Kit
JDK	Java Development Kit

Abréviation	Signification
XML	eXtensible Markup Language
LLM	Large Language Model

Table des figures

1.1	Diagramme de Gantt du projet MoviesApp	14
2.1	Diagramme de cas d'utilisation de MoviesApp	19
2.2	Diagramme d'activité — Processus Vibe Search	20
2.3	Diagramme d'activité — Processus Scan de Mood	21
2.4	Diagramme de classes — Modèle de données et adaptateurs	22
2.5	Diagramme de classes — Activités principales	22
2.6	Diagramme de classes — Fonctionnalités IA et gestion des films	23
3.1	Architecture globale de MoviesApp	24
4.1	Écrans d'authentification de MoviesApp	28
4.2	Page principale de MoviesApp	29
4.3	Page de détail d'un film	30
4.4	Gestion des films vus et favoris	31
4.5	Fonctionnalité Vibe Search	32
4.6	Fonctionnalité Scan de Mood	33

Liste des tableaux

2.1	Besoins fonctionnels de MoviesApp	16
2.2	Besoins non fonctionnels de MoviesApp	17
2.3	Règles de gestion de MoviesApp	18
3.1	Technologies Android utilisées	25
3.2	Technologies backend utilisées	25
3.3	APIs et services externes utilisés	26
3.4	Environnement logiciel de développement	27
4.1	Endpoints de l'API FastAPI	39

Table des matières

Dédicaces	1
Remerciements	2
Résumé	3
Abstract	4
Liste des Abréviations	5
Liste des Figures	7
Liste des Tableaux	8
Introduction Générale	11
1 Contexte Général du Projet	12
Introduction	12
1.1 Présentation du Projet	12
1.1.1 Problématique	12
1.1.2 Solution Proposée	12
1.2 Méthodologie de Travail et Planification	13
1.2.1 Méthodologie Adoptée	13
1.2.2 Diagramme de Gantt	13
Conclusion du Chapitre 1	14
2 Analyse et Conception	15
Introduction	15
2.1 Étude Fonctionnelle	15
2.1.1 Analyse des Besoins Fonctionnels	15
2.1.2 Analyse des Besoins Non Fonctionnels	16
2.2 Étude Conceptuelle	17
2.2.1 Règles de Gestion	17
2.2.2 Diagrammes UML	19

Conclusion du Chapitre 2	23
3 Étude Technique	24
Introduction	24
3.1 Architecture du Projet	24
3.2 Technologies Utilisées	25
3.2.1 Technologies Côté Client (Android)	25
3.2.2 Technologies Côté Serveur (Backend)	25
3.2.3 APIs et Services Externes	26
3.3 Environnement de Travail	26
3.3.1 Environnement Matériel	26
3.3.2 Environnement Logiciel	27
Conclusion du Chapitre 3	27
4 Mise en Œuvre	28
Introduction	28
4.1 Réalisation	28
4.1.1 Authentification et Inscription	28
4.1.2 Page Principale — Exploration des Films	29
4.1.3 Détail d'un Film	30
4.1.4 Gestion des Films Vus et Favoris	31
4.1.5 Fonctionnalités d'Intelligence Artificielle	32
4.2 DevOps et Déploiement	34
4.2.1 Gestion de Version avec Git et GitHub	34
4.2.2 Backend FastAPI	34
4.2.3 Firebase	34
Conclusion du Chapitre 4	34
Conclusion Générale et Perspectives	35
Références	37
Annexes	38

Introduction Générale

Le cinéma occupe une place centrale dans la vie culturelle et sociale contemporaine. Avec l'essor des plateformes numériques et la multiplication des contenus audiovisuels, les utilisateurs sont confrontés à un volume croissant de films disponibles, rendant la découverte et la gestion de ces contenus de plus en plus complexes.

Dans ce contexte, les applications mobiles jouent un rôle déterminant en offrant aux utilisateurs des outils personnalisés et intelligents pour naviguer dans cet univers cinématographique vaste. L'intelligence artificielle, en particulier, ouvre de nouvelles perspectives en permettant des recommandations personnalisées, une recherche sémantique par l'humeur, et même la détection des émotions faciales pour suggérer des films adaptés à l'état d'esprit de l'utilisateur.

C'est dans ce cadre que s'inscrit notre projet **MoviesApp**, une application mobile Android développée dans le cadre de notre 4ème année du cycle ingénieur en développement à l'EMSI Casablanca Maarif. Ce projet vise à concevoir et développer une application complète intégrant des fonctionnalités avancées de gestion de films, de recommandation par intelligence artificielle, et de géolocalisation des cinémas.

Ce rapport est structuré en quatre chapitres :

- **Chapitre 1 :** Contexte Général du Projet — présentation du projet, de la problématique, de la solution proposée et de la planification.
- **Chapitre 2 :** Analyse et Conception — étude fonctionnelle, besoins fonctionnels et non fonctionnels, règles de gestion et diagrammes UML.
- **Chapitre 3 :** Étude Technique — architecture du projet, technologies utilisées et environnement de travail.
- **Chapitre 4 :** Mise en Œuvre — réalisation de l'application, implémentation et déploiement.

1. Contexte Général du Projet

Introduction

Ce premier chapitre a pour objectif de présenter le cadre général du projet MoviesApp. Nous y décrirons la problématique à laquelle répond l'application, la solution proposée, ainsi que la méthodologie de travail adoptée et la planification du projet à travers un diagramme de Gantt.

1.1 Présentation du Projet

1.1.1 Problématique

Aujourd'hui, les utilisateurs de smartphones disposent d'un accès quasi illimité à des milliers de films via différentes plateformes. Cependant, plusieurs problèmes persistent :

- La **surcharge informationnelle** : l'abondance de films disponibles rend difficile le choix d'un film adapté à ses goûts et à son humeur du moment.
- L'**absence de centralisation** : aucune application ne permet à la fois de rechercher des films, de gérer une liste personnelle, et de recevoir des recommandations intelligentes en un seul endroit.
- Le **manque de personnalisation contextuelle** : les systèmes de recommandation classiques ne tiennent pas compte de l'état émotionnel de l'utilisateur au moment de la recherche.
- La **méconnaissance des cinémas proches** : les utilisateurs n'ont souvent pas de moyen pratique pour localiser rapidement les cinémas à proximité de leur position.

Face à ces constats, il devient nécessaire de concevoir une solution mobile innovante, intelligente et personnalisée.

1.1.2 Solution Proposée

MoviesApp est une application mobile Android qui répond à ces problématiques en proposant :

- Une **exploration de films** via l'API TMDB avec filtrage par genre, tri et recherche textuelle.

- Une **gestion personnelle** des films vus et des favoris, synchronisée avec un backend FastAPI.
- Une **recherche par vibe** : l'utilisateur décrit son humeur en langage naturel, et l'IA Gemini extrait des mots-clés pour rechercher des films correspondants.
- Des **recommandations IA** basées sur la liste des films vus par l'utilisateur.
- Une **détection de mood par caméra** : l'IA analyse le visage de l'utilisateur et recommande des films adaptés à son émotion détectée.
- Une **géolocalisation des cinémas** à proximité via Google Maps et Places API.
- Un **profil utilisateur** avec authentification sécurisée via Firebase.

1.2 Méthodologie de Travail et Planification

1.2.1 Méthodologie Adoptée

Pour la réalisation de ce projet, nous avons adopté une méthodologie agile inspirée du modèle **Scrum**. Cette approche itérative nous a permis de développer l'application par incréments successifs, en testant et validant chaque fonctionnalité avant de passer à la suivante. Les principales phases de développement ont été :

1. **Analyse des besoins** : identification des fonctionnalités et des technologies requises.
2. **Conception** : modélisation UML et définition de l'architecture.
3. **Développement** : implémentation itérative des fonctionnalités.
4. **Tests** : validation des fonctionnalités sur émulateur Android.
5. **Finalisation** : corrections, optimisations et documentation.

1.2.2 Diagramme de Gantt

La figure suivante présente le planning de réalisation du projet sur la période du 14 avril au 21 mai 2026.

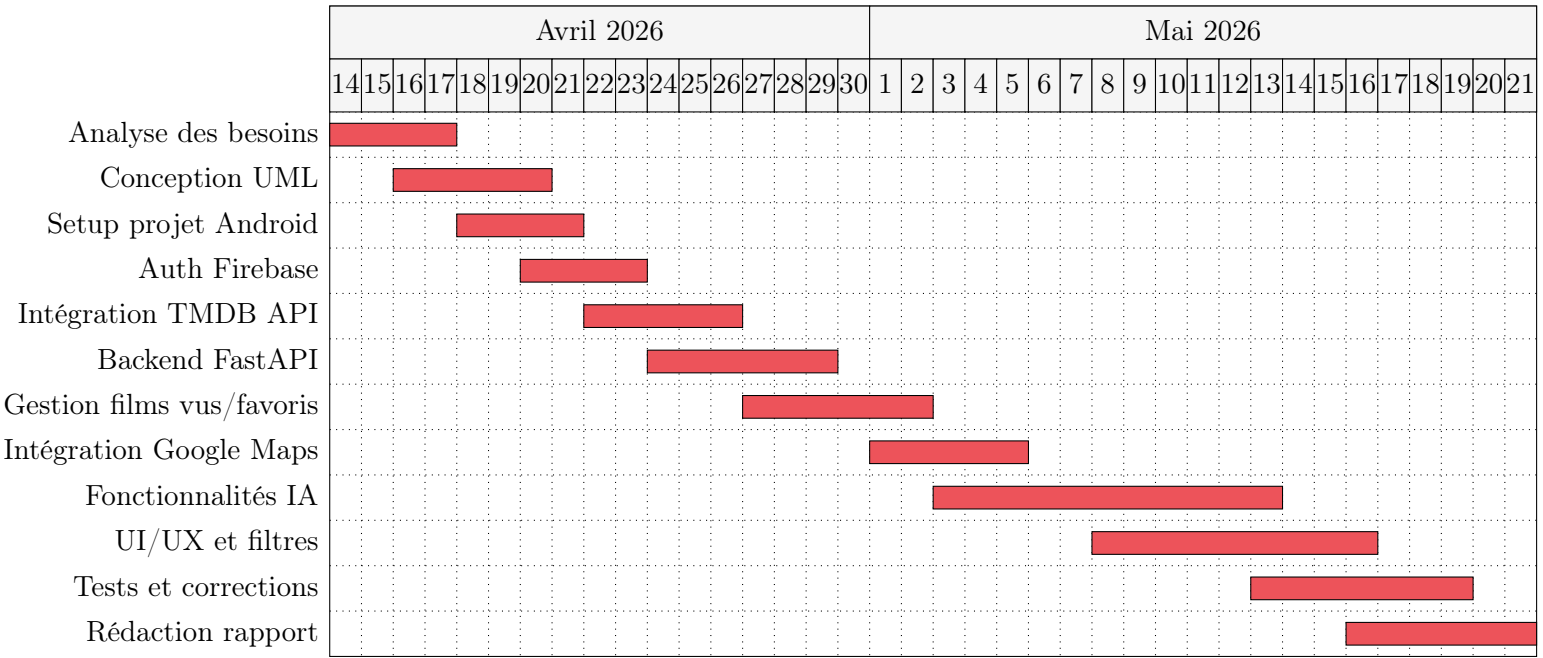


FIGURE 1.1 – Diagramme de Gantt du projet MoviesApp

Conclusion

Ce chapitre nous a permis de poser les bases du projet MoviesApp en définissant clairement la problématique, la solution proposée et la planification du travail. Le chapitre suivant sera consacré à l’analyse approfondie des besoins et à la conception de l’application.

2. Analyse et Conception

Introduction

Ce chapitre présente l'analyse fonctionnelle et conceptuelle du projet MoviesApp. Nous y détaillons les besoins fonctionnels et non fonctionnels de l'application, les règles de gestion qui régissent son fonctionnement, ainsi que les diagrammes UML qui modélisent son architecture logicielle.

2.1 Étude Fonctionnelle

2.1.1 Analyse des Besoins Fonctionnels

Les besoins fonctionnels représentent l'ensemble des fonctionnalités que l'application doit offrir à ses utilisateurs. Le tableau suivant présente ces besoins de manière structurée.

TABLE 2.1 – Besoins fonctionnels de MoviesApp

ID	Fonctionnalité	Description
BF01	Authentification	L'utilisateur peut s'inscrire et se connecter via email/mot de passe grâce à Firebase Authentication.
BF02	Exploration de films	L'utilisateur peut parcourir les films populaires, les filtrer par genre, les trier et les rechercher par titre.
BF03	Détail d'un film	L'utilisateur peut consulter les détails d'un film (titre, synopsis, note, bande-annonce).
BF04	Marquage comme vu	L'utilisateur peut marquer un film comme vu, ce qui l'ajoute à sa liste personnelle.
BF05	Gestion des favoris	L'utilisateur peut ajouter ou retirer des films de sa liste de favoris.
BF06	Recherche vocale	L'utilisateur peut effectuer une recherche de films par commande vocale.
BF07	Vibe Search	L'utilisateur décrit son humeur en langage naturel et l'IA recommande des films correspondants.
BF08	Recommandations IA	L'application génère des recommandations de films basées sur la liste des films vus.
BF09	Scan de mood	L'utilisateur prend un selfie et l'IA détecte son émotion pour recommander des films adaptés.
BF10	Carte des cinémas	L'application affiche les cinémas proches de la position de l'utilisateur sur une carte Google Maps.
BF11	Profil utilisateur	L'utilisateur peut consulter et modifier son profil.
BF12	Bande-annonce	L'utilisateur peut visionner la bande-annonce d'un film via YouTube.

2.1.2 Analyse des Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité auxquelles l'application doit se conformer.

TABLE 2.2 – Besoins non fonctionnels de MoviesApp

ID	Critère	Description
BNF01	Performance	L'application doit charger les films en moins de 3 secondes sur une connexion standard.
BNF02	Sécurité	Les données utilisateur sont protégées via Firebase Authentication et les communications s'effectuent en HTTPS.
BNF03	Disponibilité	L'application doit fonctionner sur Android 7.0 (API 24) et versions supérieures.
BNF04	Ergonomie	L'interface doit être intuitive, moderne et accessible sans formation préalable.
BNF05	Maintenabilité	Le code doit être structuré, commenté et organisé en couches distinctes.
BNF06	Extensibilité	L'architecture doit permettre l'ajout de nouvelles fonctionnalités sans refactorisation majeure.
BNF07	Fiabilité	L'application doit gérer les erreurs réseau et afficher des messages explicites à l'utilisateur.

2.2 Étude Conceptuelle

2.2.1 Règles de Gestion

Les règles de gestion définissent les contraintes métier qui régissent le comportement de l'application.

TABLE 2.3 – Règles de gestion de MoviesApp

ID	Règle	Description
RG01	Authentification obligatoire	Un utilisateur doit être authentifié pour accéder à toutes les fonctionnalités de l'application.
RG02	Film vu avant favori	Un film ne peut être marqué comme favori que s'il est d'abord marqué comme vu.
RG03	Unicité du film vu	Un utilisateur ne peut pas marquer le même film deux fois comme vu.
RG04	Condition recommandation IA	Les recommandations IA sont générées uniquement si l'utilisateur a au moins un film dans sa liste de films vus.
RG05	Connexion requise pour Vibe Search	La recherche par vibe nécessite une connexion internet active pour accéder à l'API Gemini.
RG06	Permission caméra	La fonctionnalité de scan de mood nécessite que l'utilisateur accorde l'autorisation d'accès à la caméra.
RG07	Films disponibles uniquement	Seuls les films dont la date de sortie est antérieure ou égale à la date du jour sont affichés.
RG08	Permission localisation	La fonctionnalité de carte des cinémas nécessite l'autorisation d'accès à la localisation GPS.

2.2.2 Diagrammes UML

Diagramme de Cas d'Utilisation

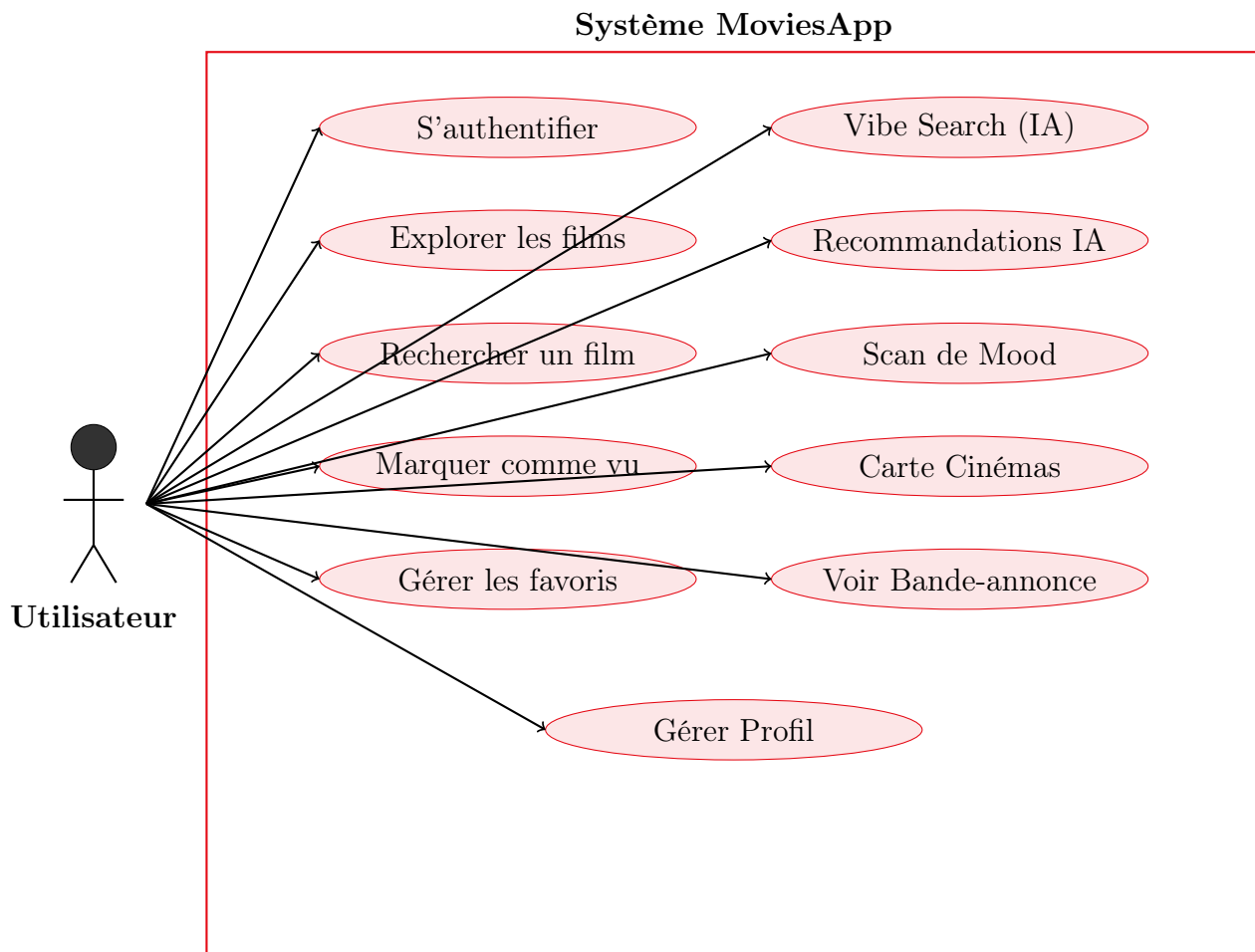


FIGURE 2.1 – Diagramme de cas d'utilisation de MoviesApp

Diagramme d'Activité — Vibe Search

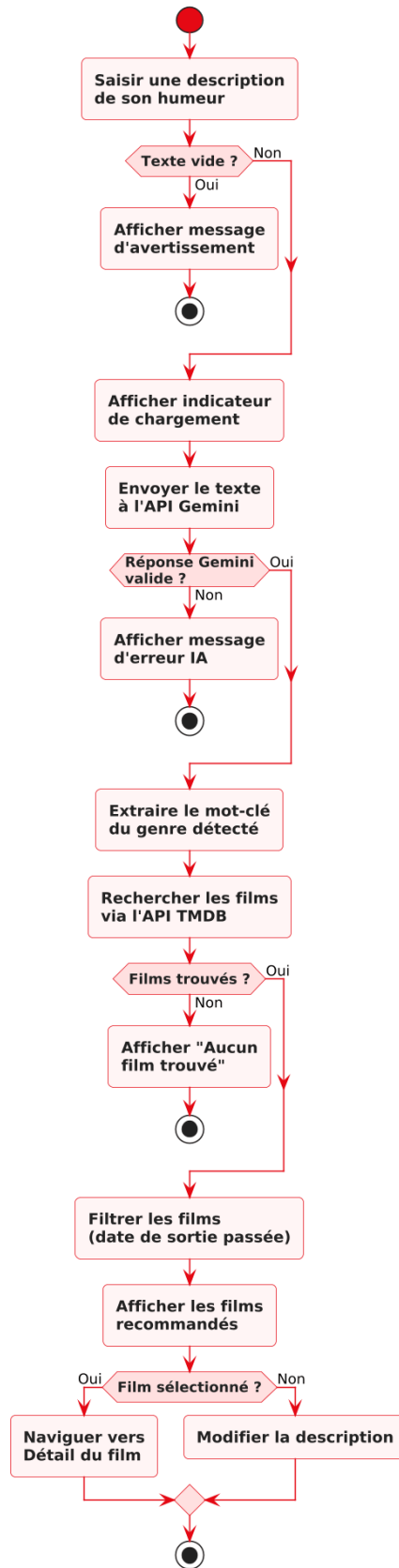


FIGURE 2.2 – Diagramme d'activité — Processus Vibe Search

Diagramme d'Activité — Scan de Mood

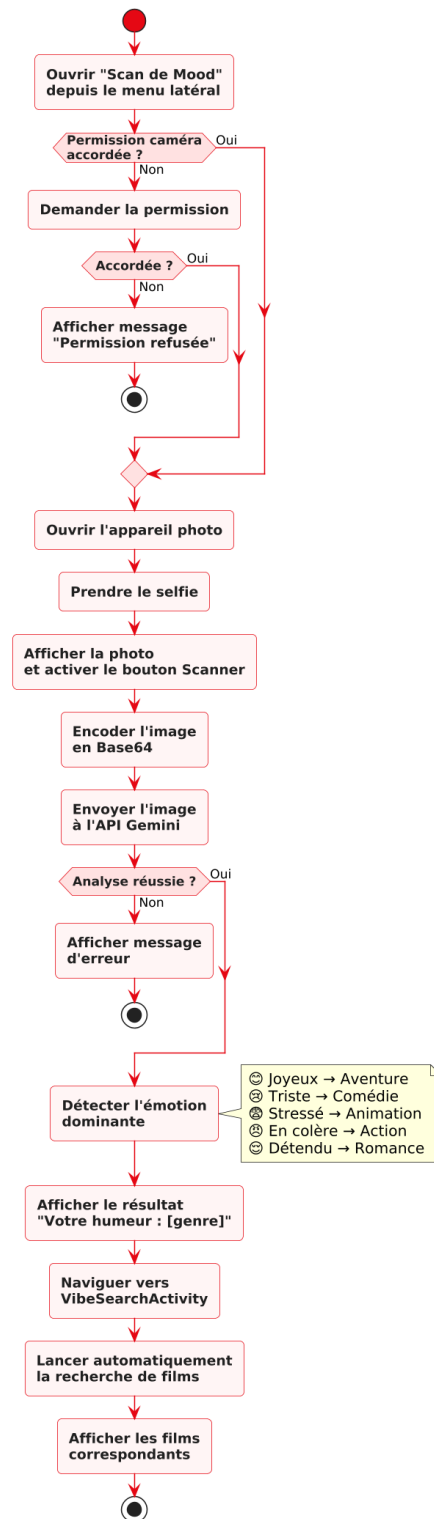


FIGURE 2.3 – Diagramme d'activité — Processus Scan de Mood

Diagramme de Classes — Modèle et Adaptateurs

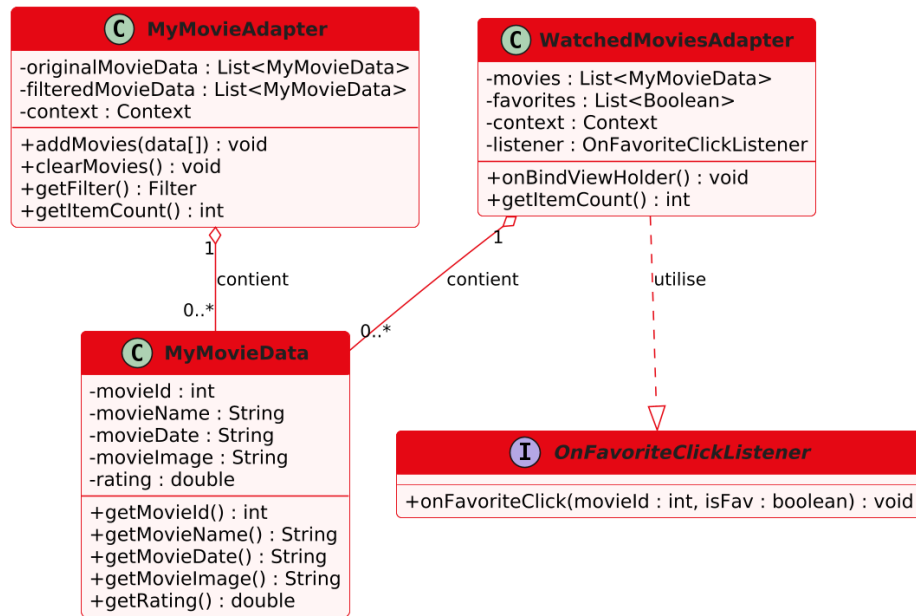


FIGURE 2.4 – Diagramme de classes — Modèle de données et adaptateurs

Diagramme de Classes — Activités Principales

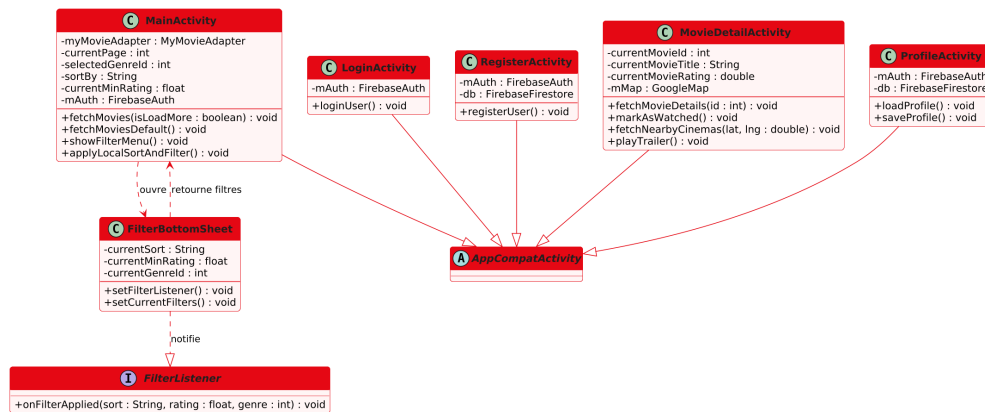


FIGURE 2.5 – Diagramme de classes — Activités principales

Diagramme de Classes — Fonctionnalités IA

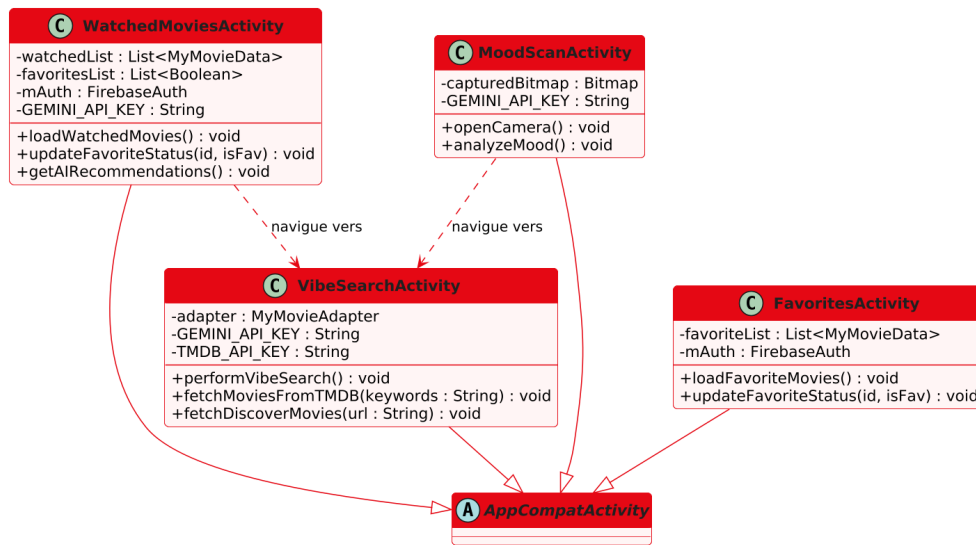


FIGURE 2.6 – Diagramme de classes — Fonctionnalités IA et gestion des films

Conclusion

Ce chapitre nous a permis de définir de manière rigoureuse les besoins fonctionnels et non fonctionnels de l'application MoviesApp, les règles de gestion qui encadrent son utilisation, et les modèles UML qui en décrivent l'architecture logicielle. Le chapitre suivant présentera les choix techniques qui ont guidé le développement de cette application.

3. Étude Technique

Introduction

Ce chapitre présente les choix techniques effectués pour la réalisation de MoviesApp. Nous y décrivons l'architecture globale du projet, les technologies et frameworks utilisés, ainsi que l'environnement de développement mis en place.

3.1 Architecture du Projet

L'application MoviesApp repose sur une architecture **client-serveur à trois tiers** :

1. **Couche Présentation (Client Android)** : l'application Android développée en Java constitue la couche de présentation. Elle gère l'interface utilisateur et les interactions avec l'utilisateur.
2. **Couche Métier (Backend FastAPI)** : le serveur FastAPI développé en Python gère la logique métier, notamment la gestion des films vus et des favoris.
3. **Couche Données** : les données sont persistées dans une base de données SQLite côté serveur, et dans Firebase Firestore pour les données utilisateur.

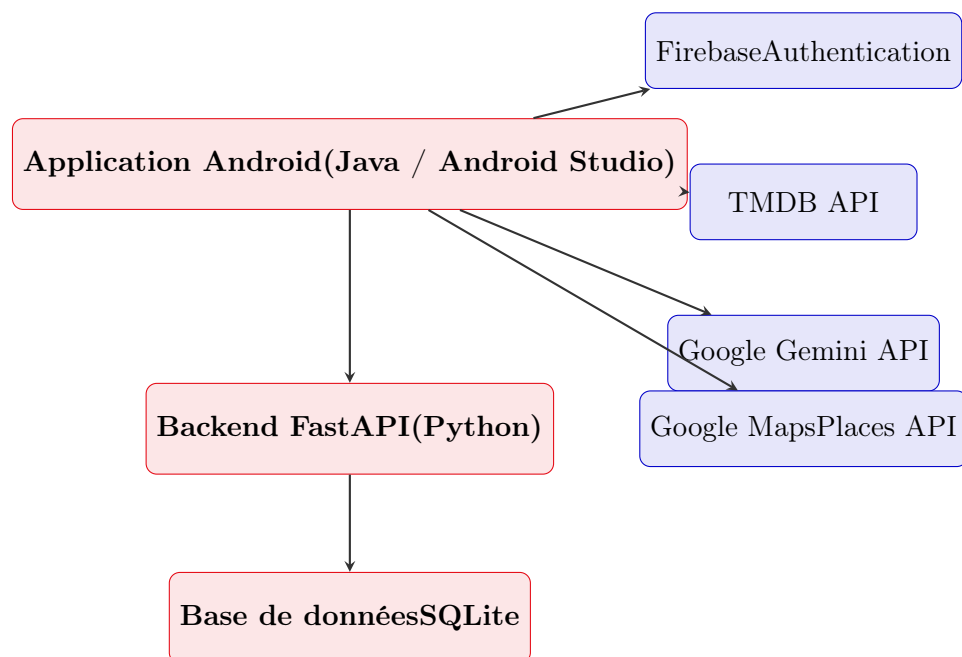


FIGURE 3.1 – Architecture globale de MoviesApp

3.2 Technologies Utilisées

3.2.1 Technologies Côté Client (Android)

TABLE 3.1 – Technologies Android utilisées

Technologie	Rôle
Java	Langage principal de développement de l'application Android.
Android Studio	IDE de développement Android (version Ladybug).
Gradle (Kotlin DSL)	Système de build et gestion des dépendances.
RecyclerView	Affichage des listes de films avec gestion efficace de la mémoire.
Material Design 3	Composants UI modernes (Chips, FAB, Bottom Sheet, etc.).
Glide	Chargement et mise en cache des images de films.
Volley	Bibliothèque HTTP pour les requêtes réseau.
OkHttp	Client HTTP pour les requêtes avancées avec gestion GZIP.
Google Maps SDK	Affichage de la carte interactive et des marqueurs.
FusedLocationProvider	Obtention de la position GPS de l'utilisateur.

3.2.2 Technologies Côté Serveur (Backend)

TABLE 3.2 – Technologies backend utilisées

Technologie	Rôle
Python	Langage de développement du backend.
FastAPI	Framework web moderne pour la création d'APIs REST performantes.
SQLite	Base de données légère pour la persistance des films vus et favoris.
Pydantic	Validation et sérialisation des données échangées via l'API.
Uvicorn	Serveur ASGI pour l'exécution de l'application FastAPI.

3.2.3 APIs et Services Externes

TABLE 3.3 – APIs et services externes utilisés

Service	Rôle
TMDB API	Base de données cinématographique : films, affiches, bandes-annonces, notes.
Firestore	Gestion sécurisée de l'authentification des utilisateurs.
Google API	Intelligence artificielle : Vibe Search, recommandations, scan de mood.
Google Maps API	Affichage de la carte interactive.
Places (New) API	Recherche des cinémas à proximité de l'utilisateur.
YouTube API	Lecture des bandes-annonces des films.

3.3 Environnement de Travail

3.3.1 Environnement Matériel

Le développement a été réalisé sur un ordinateur personnel sous **Windows 10**, avec les spécifications suivantes :

- Processeur : Intel Core i5 ou équivalent
- RAM : 16 Go
- Stockage : SSD 512 Go
- Tests : Émulateur Android (Pixel 6, API 35)

3.3.2 Environnement Logiciel

TABLE 3.4 – Environnement logiciel de développement

Outil	Version / Description
Android Studio	Ladybug — IDE principal pour le développement Android.
JDK	Java Development Kit 11 — compilation Java.
Python	Version 3.10+ — développement backend.
Git	Système de contrôle de version.
GitHub	Hébergement du code source et gestion des versions.
Postman	Test et validation des endpoints de l'API FastAPI.
Gradle	Version 9.0.1 — build system Android.

Conclusion

Ce chapitre a présenté les fondements techniques de MoviesApp, en détaillant l'architecture client-serveur adoptée, les technologies et frameworks sélectionnés, ainsi que l'environnement de développement. Ces choix ont été guidés par des critères de performance, de modernité et d'adéquation avec les besoins fonctionnels identifiés. Le chapitre suivant présentera la mise en œuvre concrète de l'application.

4. Mise en Œuvre

Introduction

Ce chapitre présente la réalisation concrète de l'application MoviesApp. Nous y décrivons les principales fonctionnalités implémentées, illustrées par des captures d'écran, ainsi que les aspects liés au déploiement et à la gestion du projet.

4.1 Réalisation

4.1.1 Authentification et Inscription

L'authentification est gérée via **Firestore Authentication**. L'utilisateur peut s'inscrire avec son nom, son email et son mot de passe, ou se connecter directement. Un mécanisme de redirection automatique vers la page principale est implémenté si l'utilisateur est déjà connecté.

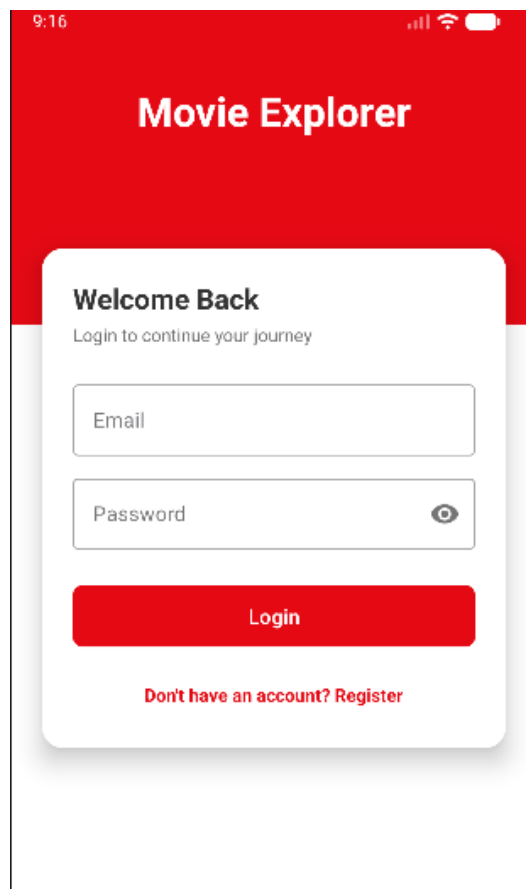


FIGURE 4.1 – Écrans d'authentification de MoviesApp

4.1.2 Page Principale — Exploration des Films

La page principale permet à l'utilisateur de parcourir les films disponibles via l'API TMDB. Elle intègre :

- Une **barre de recherche** avec recherche vocale.
- Un **bouton Vibe Search** pour la recherche sémantique par IA.
- Des **chips de genres** pour filtrer les films.
- Un **système de filtres avancés** (Bottom Sheet) permettant de trier par popularité, note ou date, et de filtrer par genre et note minimale.
- Un **menu latéral** (Navigation Drawer) donnant accès au profil, aux films vus, aux favoris et au scan de mood.

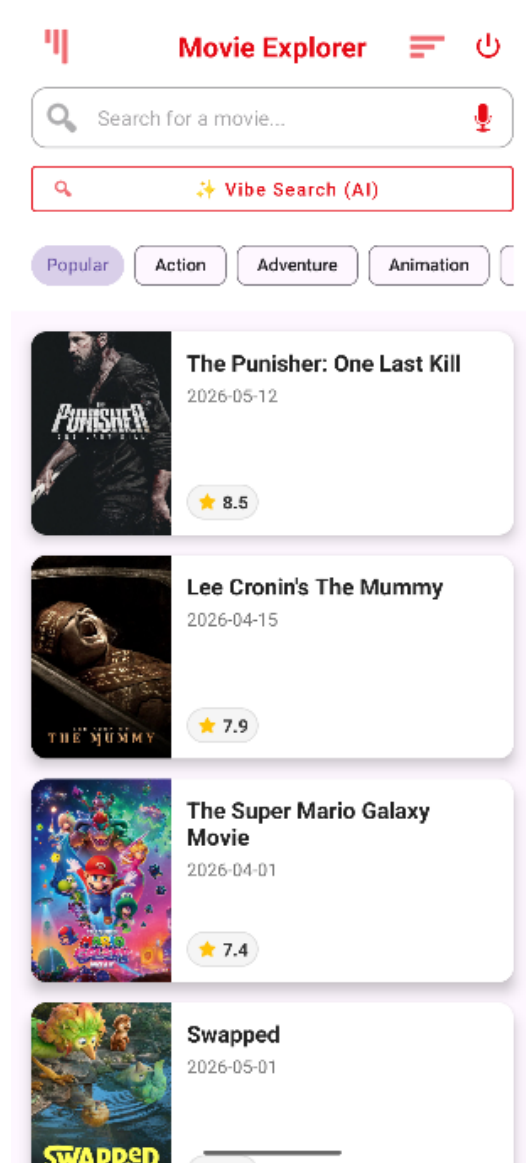


FIGURE 4.2 – Page principale de MoviesApp

4.1.3 Détail d'un Film

La page de détail d'un film affiche :

- L’affiche, le titre et le synopsis du film.
- La note moyenne des spectateurs.
- Un bouton pour lancer la bande-annonce YouTube.
- Un bouton pour marquer le film comme vu.
- Une carte Google Maps affichant les cinémas à proximité de l'utilisateur.

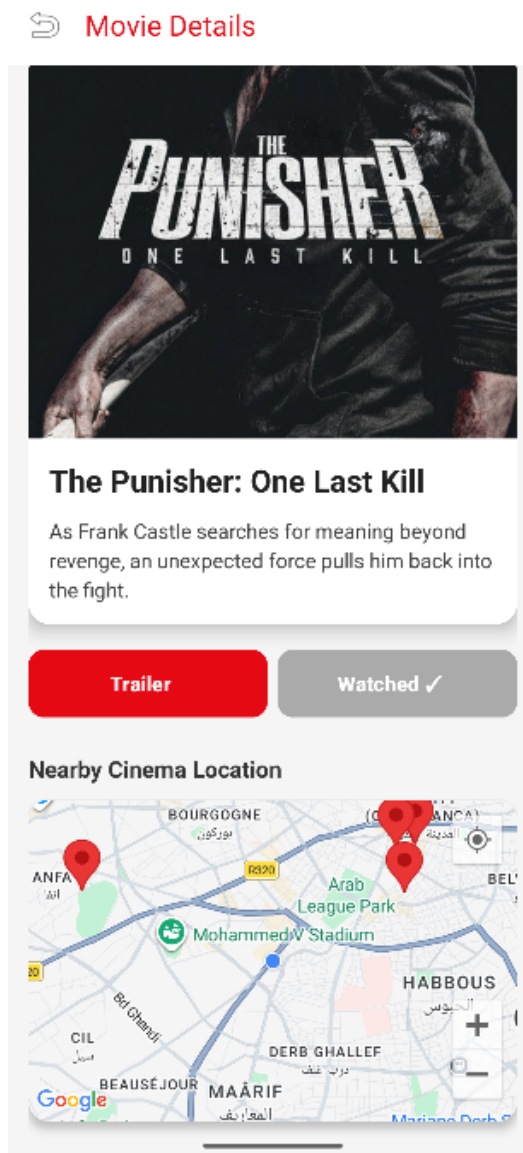


FIGURE 4.3 – Page de détail d'un film

4.1.4 Gestion des Films Vus et Favoris

L'application permet à l'utilisateur de gérer sa liste de films vus, synchronisée avec le backend FastAPI. Depuis la liste des films vus, l'utilisateur peut :

- Visualiser tous les films marqués comme vus.
- Ajouter ou retirer un film de ses favoris.
- Accéder aux recommandations IA basées sur ses films vus.

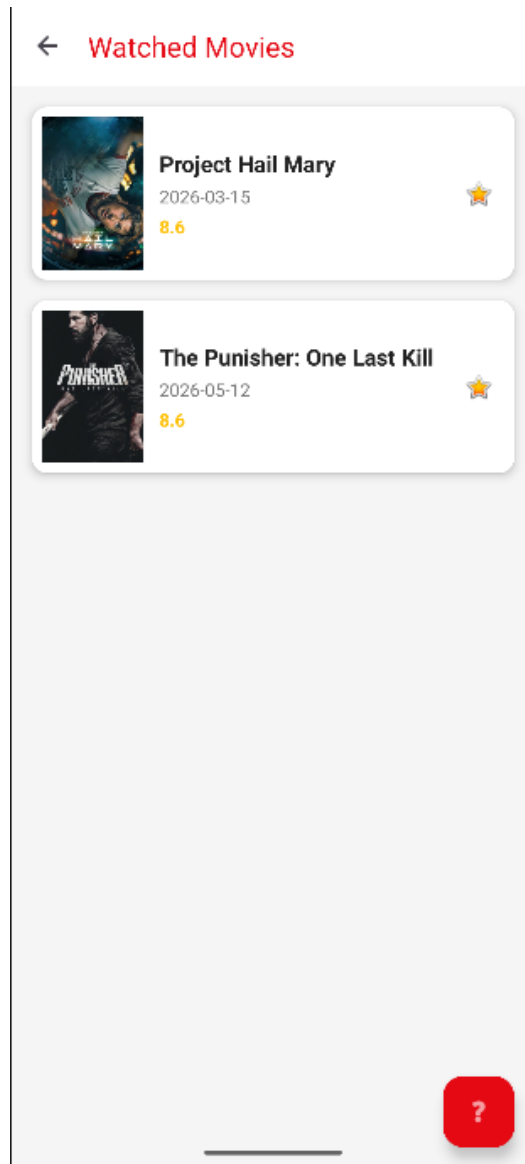


FIGURE 4.4 – Gestion des films vus et favoris

4.1.5 Fonctionnalités d'Intelligence Artificielle

Vibe Search

La fonctionnalité Vibe Search permet à l'utilisateur de décrire son humeur ou ses envies en langage naturel. L'API Google Gemini analyse cette description et extrait des mots-clés pertinents, qui sont ensuite utilisés pour rechercher des films correspondants via l'API TMDB.

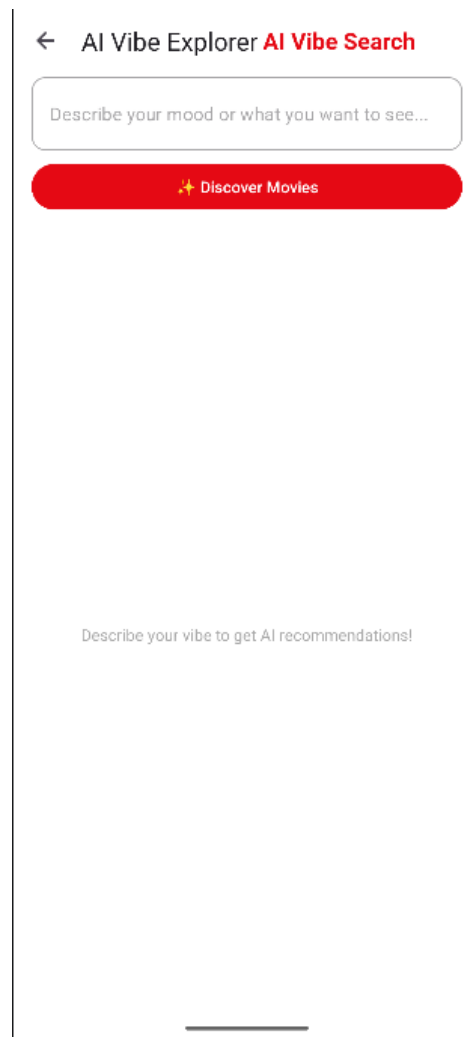


FIGURE 4.5 – Fonctionnalité Vibe Search

Recommandations IA basées sur les Films Vus

Depuis la liste des films vus, l'utilisateur peut accéder à des recommandations personnalisées. L'API Gemini analyse les titres et notes des films vus et navigue vers la page Vibe Search avec un mot-clé représentatif du goût de l'utilisateur.

Scan de Mood par Caméra

La fonctionnalité Scan de Mood permet à l'utilisateur de prendre un selfie. L'image est envoyée à l'API Google Gemini (modèle multimodal) qui analyse l'expression faciale et détecte l'émotion dominante. En fonction de cette émotion, un genre de film est recommandé et la recherche est automatiquement lancée.

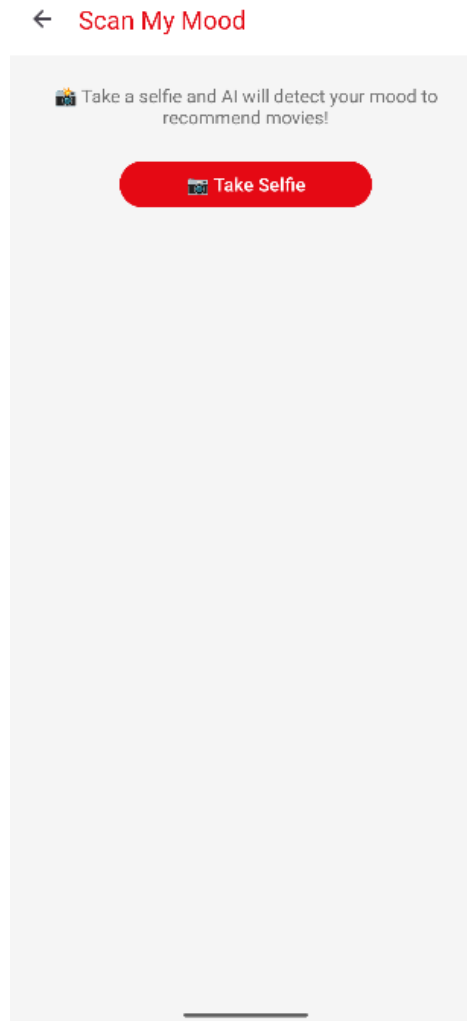


FIGURE 4.6 – Fonctionnalité Scan de Mood

4.2 DevOps et Déploiement

4.2.1 Gestion de Version avec Git et GitHub

L'ensemble du code source de l'application a été versionné à l'aide de **Git** et hébergé sur **GitHub**. Cette pratique a permis de maintenir un historique des modifications, de travailler de manière organisée et de faciliter la traçabilité des changements apportés tout au long du développement.

4.2.2 Backend FastAPI

Le backend FastAPI est exécuté localement via **Uvicorn**. Les endpoints REST exposés permettent à l'application Android de :

- Récupérer la liste des films vus d'un utilisateur (GET /watched/{user_id})
- Ajouter un film à la liste des vus (POST /watched/{user_id})
- Mettre à jour le statut favori d'un film (PATCH /watched/{user_id}/{movie_id})
- Récupérer et sauvegarder le profil utilisateur (GET/POST /profile/{user_id})

4.2.3 Firebase

Firebase Authentication est utilisé pour sécuriser l'accès à l'application. **Firebase Firestore** est utilisé pour stocker les informations de profil utilisateur de manière persistante dans le cloud.

Conclusion

Ce chapitre a présenté la réalisation concrète de l'application MoviesApp, en décrivant l'ensemble des fonctionnalités implémentées et les aspects liés au déploiement. L'application intègre avec succès des technologies modernes telles que l'intelligence artificielle, la géolocalisation et une architecture client-serveur robuste.

Conclusion Générale et Perspectives

Ce projet de fin d'année nous a permis de concevoir et développer MoviesApp, une application mobile Android intelligente dédiée à la découverte et à la gestion de films. À travers ce travail, nous avons mobilisé un ensemble de compétences techniques variées, allant du développement mobile Android en Java, à la création d'une API REST avec FastAPI, en passant par l'intégration de services d'intelligence artificielle via Google Gemini.

Les principaux apports de ce projet sont :

- La maîtrise du développement d'applications Android complètes avec une architecture claire et maintenable.
- L'intégration réussie de l'intelligence artificielle pour trois cas d'usage distincts : la recherche sémantique, les recommandations personnalisées et la détection d'émotions.
- Le développement d'un backend REST avec FastAPI, assurant la persistance des données utilisateur.
- L'utilisation de services cloud modernes (Firebase, TMDB, Google Maps) dans un contexte de production réel.
- La gestion de projet en mode agile avec planification et suivi des tâches.

Perspectives d'amélioration :

Plusieurs améliorations pourraient être envisagées pour enrichir davantage l'application :

- **Déploiement cloud du backend** : migrer le serveur FastAPI vers une plateforme cloud (AWS, Azure, Heroku) pour assurer une disponibilité permanente.
- **Système de notation** : permettre aux utilisateurs de noter les films vus et d'afficher les notes communautaires.
- **Partage social** : intégrer des fonctionnalités de partage de listes de films sur les réseaux sociaux.
- **Mode hors-ligne** : implémenter un cache local permettant l'utilisation de l'application sans connexion internet.
- **Recommandations collaboratives** : développer un algorithme de filtrage collaboratif basé sur les préférences de plusieurs utilisateurs.

- **Version iOS** : porter l'application sur la plateforme iOS en utilisant Flutter ou React Native.

Ce projet a constitué une expérience enrichissante qui nous a permis d'appliquer concrètement les connaissances acquises durant notre formation à l'EMSI et de développer notre autonomie dans la conduite d'un projet informatique complet.

Références

1. TMDb API Documentation. *The Movie Database API v3*. [En ligne]. Disponible sur : <https://developers.themoviedb.org/3>
2. Google. *Gemini API Documentation*. [En ligne]. Disponible sur : <https://ai.google.dev/docs>
3. Google. *Firebase Documentation*. [En ligne]. Disponible sur : <https://firebase.google.com/docs>
4. FastAPI. *FastAPI Documentation*. [En ligne]. Disponible sur : <https://fastapi.tiangolo.com>
5. Android Developers. *Android Developer Documentation*. [En ligne]. Disponible sur : <https://developer.android.com>
6. Google. *Google Maps Platform Documentation*. [En ligne]. Disponible sur : <https://developers.google.com/maps>
7. Google. *Places API (New) Documentation*. [En ligne]. Disponible sur : <https://developers.google.com/maps/documentation/places/web-service/overview>
8. Bumptech. *Glide Image Loading Library*. [En ligne]. Disponible sur : <https://github.com/bumptech/glide>
9. Square. *OkHttp Documentation*. [En ligne]. Disponible sur : <https://square.github.io/okhttp>
10. Material Design. *Material Design 3 Guidelines*. [En ligne]. Disponible sur : <https://m3.material.io>

Annexes

Annexe A — Structure du Projet Android

Listing 4.1 – Structure des fichiers principaux du projet

```
1 MoviesApp/  
2 |-- app/  
3 |   |-- src/main/  
4 |       |-- java/com/example/moviesapp/  
5 |           |-- MainActivity.java  
6 |           |-- LoginActivity.java  
7 |           |-- RegisterActivity.java  
8 |           |-- MovieDetailActivity.java  
9 |           |-- WatchedMoviesActivity.java  
10 |           |-- FavoritesActivity.java  
11 |           |-- ProfileActivity.java  
12 |           |-- VibeSearchActivity.java  
13 |           |-- MoodScanActivity.java  
14 |           |-- MyMovieAdapter.java  
15 |           |-- WatchedMoviesAdapter.java  
16 |           |-- MyMovieData.java  
17 |           |-- FilterBottomSheet.java  
18 |       |-- res/  
19 |           |-- layout/  
20 |           |-- menu/  
21 |           |-- values/  
22 |       |-- AndroidManifest.xml  
23 |-- gradle/  
24 |   |-- libs.versions.toml  
25 |-- build.gradle.kts
```

Annexe B — Endpoints de l'API FastAPI

TABLE 4.1 – Endpoints de l'API FastAPI

Méthode	Endpoint	Description
GET	/watched/{user_id}	Récupérer la liste des films vus
POST	/watched/{user_id}	Ajouter un film à la liste des vus
PATCH	/watched/{user_id}/{movie_id}	Mettre à jour le statut favori
GET	/profile/{user_id}	Récupérer le profil utilisateur
POST	/profile/{user_id}	Sauvegarder le profil utilisateur

Annexe C — Permissions Android

Listing 4.2 – Permissions déclarées dans AndroidManifest.xml

```
1 <uses-permission android:name="android.permission.INTERNET" />
2 <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"
  " />
3 <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"
  " />
4 <uses-permission android:name="android.permission.
  ACCESS_COARSE_LOCATION" />
5 <uses-permission android:name="android.permission.RECORD_AUDIO" />
6 <uses-permission android:name="android.permission.CAMERA" />
7 <uses-feature android:name="android.hardware.camera"
8           android:required="false" />
```